

Neel Suresh, Aman Deva, Ruben Vipinraj, Clarissa Carrillo, Hima Nagi Reddy, Sanjana Shangle
Professor Prakash
CS4485
11 December 2025

Path Based Prediction for 360° Video Streaming

Problem statement

Streaming 360° video often comes with many challenges. First, there are massive bandwidth requirements when streaming in high resolution such as 8k or 12k. Furthermore, due to the nature of 360° video, a viewer can only view a fraction of the full screen, making it inefficient to always stream the full 360° view.

Goals/Objective

The goal of this project is to optimize 360° video streaming by predicting where viewers will look next based on their past head movements. This approach, known as path-based prediction, aims to only stream the relevant portion of the screen to avoid transmitting the full 360° view. The main objective is to reduce the bandwidth needed to stream 360° video while maintaining a positive user experience. This paper explores the various approaches and techniques we have leveraged to achieve this goal.

Major ideas

The dataset tracks 48 different users as they watch 18 different 360° videos. The videos span 5 different categories: performance, sport, film, documentary, and talk show. The dataset tracks each user as they watch the videos, recording their head direction as quaternion values(w, x, y, z) as well as their head position(x, y, z) at each timestamp.

The preprocessing starts by loading the dataset and resampling the data to remove duplicate entries and ensure that successive data entries increase by consistent intervals of time. For each data entry, the quaternion values are converted to Euler values(roll, pitch, yaw), as these are more intuitive and simpler to work with. We decided to discard roll as a feature as this does not affect the viewer's head direction. Instead, we only use pitch and yaw as these are enough to accurately record the viewer's head direction. Pitch and yaw are recorded as their sine and cosine counterparts to avoid issues with angle wrap. Finally, velocity and acceleration are calculated for the pitch and yaw as this information is valuable in training the model. The finalized dataset after preprocessing contains 8 features: $\cos(\text{yaw})$, $\sin(\text{yaw})$, $\cos(\text{pitch})$, $\sin(\text{pitch})$, yaw velocity, pitch velocity, yaw acceleration, and pitch acceleration.

Solution approach

- **LSTM:**

The LSTM model showed consistent improvement across epochs. The training cosine loss decreased from approximately 0.017 in the first epoch to about 0.007 with the final epoch. Meanwhile, the validation cosine loss improved from roughly 0.0128 to 0.0104 over that same

period. Now, angular-error metrics had a similar downward trend where the training error dropped from around 4.2° to 2.2° and validation error decreased from approximately 3.2° – 3.3° to about 2.3° . The learning rate remained constant at $2.5e-4$ throughout training and the radian-loss logs stayed near zero. Overall, the LSTM shows stable convergence and solid capability in modeling head-movement sequences which makes it a reliable recurrent baseline for our sequential prediction pipeline.

During development, we encountered some `state_dict` mismatch errors when testing a saved checkpoint. This checkpoint corresponded to a GRU model with a hidden size 96. Now, because the GRU and LSTM architectures use different internal gating structures, this makes their weight matrices incompatible. Consequently, this prevented the LSTM from loading the GRU weights which required us to retrain the LSTM from scratch with correctly initialized parameters this time.

- **GRU:**

The GRU model is an improvement over the LSTM. It only contains two gates, the update gate and the reset gate, making it more efficient to train and run. Additionally, it solves the exploding/vanishing gradient problem that LSTM's may face. For these reasons, we wanted to train a GRU model to compare with the results of the LSTM.

Similar to the LSTM, as each epoch passed, we noticed a steady decrease in the cosine loss which we used for training the model. We noticed that around 30 epochs, the model leveled out, no longer benefiting greatly by additional epochs, for this reason, we didn't train beyond 30 epochs. Performance of this model on the test set can be seen later in the report, however as a brief summary, GRU does perform slightly better than the LSTM model overall.

We also experimented with applying a Kalman filter to the GRU model before training to smooth the yaw and pitch measurements. The idea behind this was that slight inaccuracies in the sensor might cause the VR headset data to appear noisy. By applying a Kalman filter, we could ideally avoid any sudden spikes and produce a cleaner dataset that would be easier for the GRU model to train on. In practice, however, we found that the Kalman filter provided questionable results and often performed worse than using a GRU model without a Kalman filter. It is likely that the Kalman filter was over-smoothing the data and removing important patterns in the data for the GRU model to learn.

Input Features: yaw, pitch, $\cos(\text{yaw})$, $\sin(\text{yaw})$

Sequence Length: As defined in the preprocessing pipeline

Output: Predicted 3-D head orientation vector

Although the LSTM produced stable improvements and achieved reasonable accuracy, it did **not** outperform the GRU baseline. The GRU demonstrated slightly better angular-error performance and faster responsiveness to directional changes.

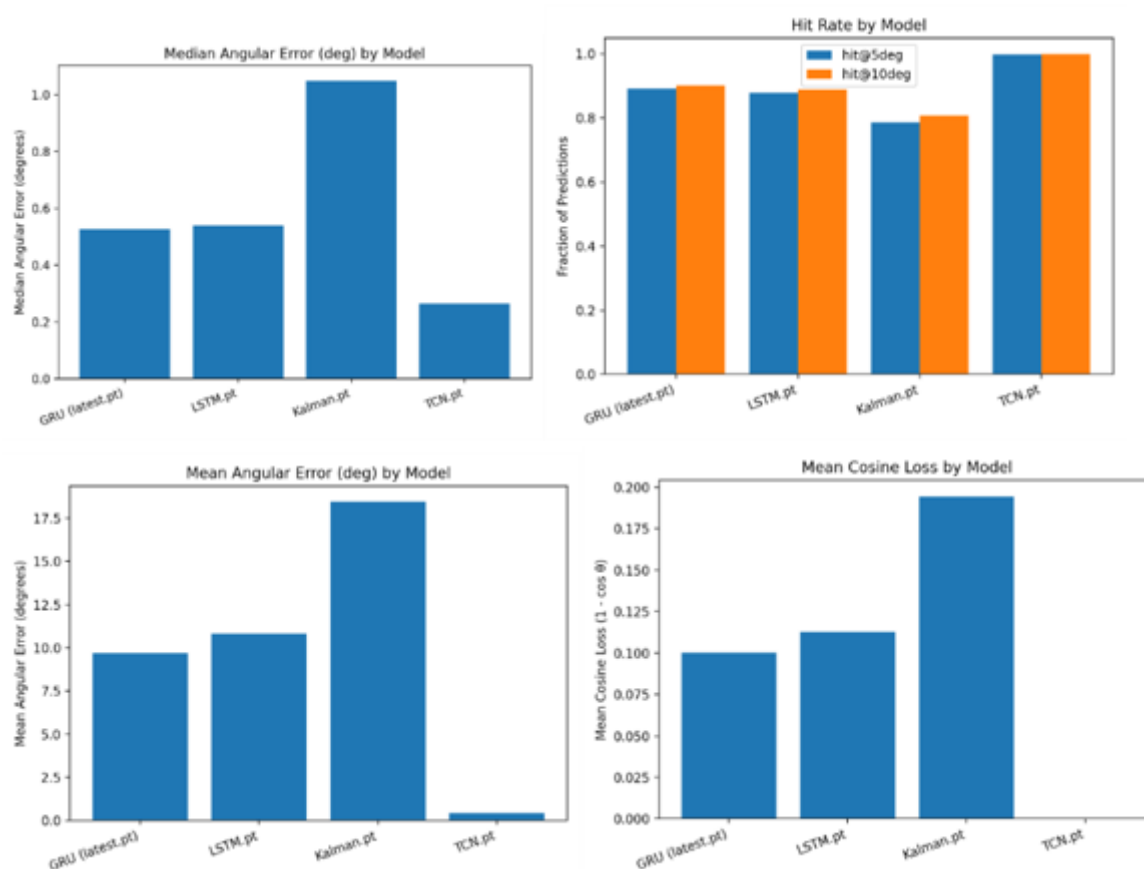
- **TCN**

The TCN model relies on three core ideas, causal convolutions, dilated convolutions, and residual connections. The combination of these three ideas created the best model out of the GRU and LSTM. Causal convolutions allow the model to make sure it only accesses

information from the past and prevents future peeking. Dilated convolutions space out the convolution filters so the model can see far back into the sequence giving it a much larger receptive field and allowing it to model multi-second motion patterns. Lastly, the residual connections help it improve gradient flow and make training more efficient compared to the LSTM and GRU models.

TCN performed exceptionally well for this problem because the head movement that humans possess contain both smooth, gradual trends, and short bursts of motion. These filters that were built into the TCN model were perfect to detect these patterns and predict user head movement. As a result, the TCN model achieved the lowest angular errors and the highest hit rates among the other models that we tested.

Final outcome evaluation/metrics



Challenges

We faced weight-loading errors when attempting to test saved checkpoints across different architectures (such as loading a GRU checkpoint into an LSTM). Because GRU and LSTM maintain fundamentally different gating mechanisms, their internal weight matrices are incompatible. Another challenge was with the TCN model training time. Unlike LSTM and GRU models, the TCN architecture looks back through the entire history for every single timestep. Because our data runs at 60 frames per second, even predicting just one frame ahead required almost 40 hours of training. When we extended that window to 2.5 seconds, or about 150

frames, the estimated training time jumped to roughly 916 hours, which is nearly 38 days. This made full TCN training impractical within our project timeline.

Opportunities for future enhancement

- We would like to improve the Kalman filter for the GRU model in hopes to make the quality for testing to be better than in the past.
- In the future, we would like to train the TCN model to predict longer in the future. Currently, the TCN model successfully predicts 1 frame into the future but due to long training times, this could not be replicated for 150 frames or 2.5 seconds into the future. After letting the model run for about 70 hours, we predict that it would take around 916 hours to fully train the model for the 2.5 second mark hence why we would like to improve upon that in the future.